

Project 1 — Intern Working Guide

Learning from Sparse, Skewed, or Irregular Data

IITB — Optiver AI Innovation Lab | July — November 2026 | PI: Prof. Siddhartha Duttagupta

Welcome — Read This First

This document is your complete instruction set. It tells you what the project is, which datasets to use, what to build for each research theme, the exact steps to follow, and what to hand in. You should not need any other document to start working.

The project in one paragraph: Banking data breaks the assumptions normal ML makes.

Transactions arrive with long irregular gaps, new fraud types have almost no labelled examples, data fields are often missing, and fraud itself is under 1% of records. We are building and benchmarking five models — one per failure mode — on public financial datasets, and writing a technical report on what works.

How to treat the recommended models: each theme below has a recommended approach — it is the default and you should start there. But if you find an alternative that is **simpler or performs better**, you may propose it: show a quick comparison on the validation set, get a yes from the weekly meeting, then switch. What is **not** negotiable: the datasets, the fixed data splits, the evaluation metrics, and the deadlines.

Ground Rules (apply to everything)

- R1.** All results must come from the shared **evaluate()** harness (built in Week 1). No hand-computed metrics.
- R2.** Use the fixed train/validation/test splits (seed 42) saved in `data/splits/`. Never re-split, never touch test data until the end.
- R3.** Fit anything learnable — scalers, imputers, resamplers, generators — on training data only. This is the #1 way results get invalidated.
- R4.** Never report plain accuracy. Our metrics: AUROC, F1, AUPRC, recall (fraud class).
- R5.** One config file per experiment, committed to the repo. Anyone must be able to re-run your result with one command.
- R6.** Stuck for more than a day? Raise it in the group chat, don't sit on it. Negative results are fine; silent delays are not.
- R7.** Every Friday: update the shared results table + write 5 lines (what ran, what worked, what's next).

Step 0 — Setup & Datasets (everyone, Week 1)

Datasets

Dataset	Where	Size / fraud rate	Used by themes
PaySim	Kaggle: 'PaySim1'	6.3M transactions, ~0.13% fraud	1.1, 1.2, 1.5
IEEE-CIS Fraud	Kaggle: 'ieee-fraud-detection'	590K transactions, ~3.5% fraud, many missing columns	1.1, 1.2, 1.3, 1.5
UCI Credit Default	UCI ML Repository	30K accounts, 22.1% default	1.4, 1.5 (reference — baselines already done: XGBoost AUROC 0.774)
MIMIC-III	physionet.org (access granted)	~46K patients, multi-table	1.3

Setup steps

1. Clone the repo; create the environment: Python 3.10+, PyTorch, XGBoost, scikit-learn, imbalanced-learn, pandas.
2. Download the datasets into **data/raw/**. Raw files are read-only — never edit them.
3. Run (or write, if first) the loader scripts → clean parquet files in **data/processed/** with a label column **y**.
4. Create/load the fixed 70/15/15 stratified splits (seed 42) in **data/splits/**.
5. For PaySim and IEEE-CIS, build the **sequence version**: per customer, events in time order, each with its time-gap from the previous event.
6. Build **evaluate(y_true, y_scores)** → AUROC, F1, AUPRC, recall; and **run_experiment(config)** which trains, evaluates and appends one row to results.csv. Sanity check: re-run the UCI baseline — you must reproduce AUROC ≈ 0.774.

What you hand in: working environment, processed datasets, splits, harness verified against the UCI number.

Theme 1.5 — Extreme Class Imbalance (Week 2)

Goal: fraud is <1% of data; a model predicting 'never fraud' is 99% accurate and useless. Find the training recipe that makes models care about the rare class — this recipe is then used by all other themes.

Recommended approach: imbalance-aware training — focal loss / cost-sensitive (class-weighted) losses. This is a training recipe, not a new model.

Steps

1. Train XGBoost and a small MLP on PaySim, IEEE-CIS and UCI — three ways each: (a) plain, (b) class weights (scale_pos_weight / class_weight='balanced'), (c) focal loss for the MLP (gamma=2; try 1 and 5).
2. Optionally add SMOTE as a fourth variant (training data only!) for the comparison.

3. Tune the decision threshold on validation data for best F1.

4. Produce one comparison table (method x dataset x metrics). The winner becomes the **standard loss** for themes 1.1–1.4.

Allowed alternatives: ADASYN / ensemble methods (EasyEnsemble, RUSBoost) if you can show they beat focal/weighted losses on validation.

What you hand in: comparison table + half-page summary declaring the standard loss.

Theme 1.1 — Sparse Event Sequences (Weeks 3–6)

Goal: customers transact in bursts with long silent gaps. The gap itself is signal (dormancy then sudden activity is a classic fraud pattern). Build a sequence model where time-gaps are a first-class input. **This encoder is the backbone that themes 1.2 and 1.4 reuse — build it well.**

Recommended approach: Transformer sequence encoder with time-gap positional encoding. View the customer's risk as a latent state updated per event.

Steps

1. Input per customer: last N=100 events. Each event = its features + its time-gap since the previous event. Pass the time-gap (log-scaled) through a small embedding and add it to the event embedding.
2. Baseline first: XGBoost on summary features (event counts, mean/max gap, time since last event). Your Transformer must beat this to count.
3. Model: 2-layer Transformer encoder, 64–128 dims, prediction head for fraud probability. Train with the standard loss from Theme 1.5.
4. Evaluate on held-out customers, and separately on **dormant** customers (long gap before the prediction point) — the dormant slice is the headline number.
5. One ablation: the same model with time-gap input removed. The performance difference is the evidence that modelling irregular time matters.
6. Save the trained encoder weights to **models/encoder_11.pt** — other themes load this file.

Allowed alternatives: GRU/LSTM with time-delta embeddings if it matches the Transformer on validation (it is simpler); Neural Hawkes only if you have spare time and a clear reason.

What you hand in: results vs baseline, dormant-slice results, ablation table, saved encoder weights.

Theme 1.2 — Few-Shot / Zero-Shot Risk Prediction (Weeks 5–8)

Goal: a new fraud type appears with only 5–20 confirmed examples. Learn a similarity space where a new case is scored by closeness to those few examples — no retraining for every new type.

Recommended approach: prototypical network on top of the Theme 1.1 embeddings.

Steps

1. Define episodes first: split fraud types — most for training, a few held out as 'new'. Fix 200 test episodes (k = 5, 10, 20 support examples each) with a seed, so every method sees identical episodes.
2. Load **models/encoder_11.pt**; embed every transaction sequence. No new architecture needed.
3. Prototype = mean embedding of the k examples of the new type. Classify queries by distance to the nearest prototype. This is ~50 lines of code.
4. Baseline: k-NN on raw features using the same k examples.
5. Zero-shot probe: score new-type cases purely by embedding distance from normal behaviour (no labels at all).
6. Report performance vs k with mean $\pm$ std over the 200 episodes. If the baseline ties the prototype model, say so honestly.

Allowed alternatives: light contrastive fine-tuning of the backbone if prototypes underperform; Siamese networks. Avoid MAML — fiddly, and not worth the time here.

What you hand in: accuracy/AUPRC vs k table with error bars, baseline comparison, zero-shot result.

Theme 1.3 — Missing Features in Multimodal Data (Weeks 9–10)

Goal: real records arrive with whole channels absent — KYC not uploaded, demographics withheld. Predictions must stay stable when a modality is missing. Note: sometimes absence itself is informative.

Recommended approach: multimodal transformer — per-modality encoders, cross-modal attention fusion, and **modality dropout** during training to simulate missing inputs.

Steps

1. Group features into 2–3 modalities: MIMIC-III → demographics / labs / notes; IEEE-CIS → transaction / identity.
2. One small encoder (MLP is fine) per modality → one embedding each → a 1-layer cross-modal attention block fuses them → prediction head. Standard loss from 1.5.
3. **Modality dropout** — the key trick: during training, randomly replace a whole modality with a learned 'missing' token ($p=0.3$). The model learns to lean on whatever channels remain.
4. Also add per-feature missingness indicator columns (1 = absent) — cheap and captures informative absence.
5. Baseline: XGBoost fed raw NaNs (it handles them natively and is strong).
6. Evaluate: each modality removed one at a time, plus 20/40/60% random feature missingness. Plot performance vs missingness per method — this plot is the headline figure.

Allowed alternatives: MICE or simple imputation + any classifier, if it beats modality dropout on validation; BERT-style per-feature masking if you're comfortable with it.

What you hand in: degradation plot, modality-removal table, comparison vs XGBoost-NaN baseline.

Theme 1.4 — Generative Reconstruction (Weeks 11–12)

Goal: customers with short or sparse histories have noisy, unstable representations. Use a generative model to denoise them into stable ones, and check whether that helps downstream prediction.

Recommended approach: VAE trained on the learned Theme 1.1 embeddings (not raw features). Reconstruction acts as a regulariser — the denoising counterpart of filtering in the PI's sensing research.

Steps

1. Load embeddings for all customers from **models/encoder_11.pt**.
2. Small VAE: 2-layer encoder → 16–32 dim latent → 2-layer decoder, trained to reconstruct the embeddings.
3. Test (a): classify from the denoised (reconstructed) embedding vs the raw embedding — compare especially on customers with short/sparse histories.
4. Test (b): fine-tune the 1.1 encoder with the VAE reconstruction loss added as a regulariser — does robustness improve?
5. Report both through the harness. A clear negative result ('denoising did not help') is an acceptable outcome.

Allowed alternatives: CTGAN (SDV library) for synthetic minority oversampling instead — evaluated through the Theme 1.5 pipeline next to SMOTE — if the VAE route stalls. Skip diffusion models; too

costly for this window.

What you hand in: comparison tables for tests (a) and (b), short written conclusion.

Schedule at a Glance (16 weeks)

Weeks	Work	Done means
1	Step 0: setup, datasets, splits, harness	UCI number reproduced through harness
2	Theme 1.5	Standard loss declared
3–6	Theme 1.1 (backbone)	Encoder saved, dormant-slice + ablation results in table
5–8	Theme 1.2 (overlaps 1.1 once encoder exists)	vs-k results with error bars
9–10	Theme 1.3	Degradation plot done
11–12	Theme 1.4	Both tests reported
13–14	Final runs + one ablation per theme	Master results table frozen
15	Technical report + repo cleanup	Draft complete, all configs committed
16	Buffer, presentation, submission	Project delivered

If You Fall Behind

Shrink the experiment, never the theme: use one dataset instead of two, one k instead of three, smaller N. Every theme must produce at least one honest benchmarked result. If a recommended model is not working by the end of its window, log the negative result, fall back to the listed alternative or the baseline, and move on.

What the Project Delivers (mid-November 2026)

- [] Five themes benchmarked through one shared harness, one master results table
- [] Saved backbone encoder + per-theme heads, reproducible from config files
- [] Headline figures: 1.1 time-gap ablation, 1.2 accuracy-vs-k, 1.3 degradation plot, 1.4 denoising comparison, 1.5 loss comparison
- [] Technical report (one section per theme) + final presentation

Questions: raise them in the weekly meeting or the group chat. When in doubt, choose the simpler option and write down why.